Timewalk alignment: the parabola method

How the scripts work and how they connect
CMS IT Phase-2 quad module, 4 CROC chips (Ids 0–3)

The goal: line up the pixel geometries

Edge and corner pixels are bigger. More input capacitance means a slower preamp, so their **threshold vs. CAL_EDGE_FINE_DELAY** dip sits at a different delay than the Standard pixels.

Raising a **DAC_PREAMP_*_LIN** register speeds that preamp up and moves its curve **later**.

The code tunes those DACs until every target curve sits within **MAX_ABS_SHIFT** of the reference.

Geometry	Size [µm]	Chips 0/2	Chips 1/3
Standard (ref)	25 × 100	M	M
Long_Row	25 × 225	T	T
Long_Col	87.5 × 100	L	R
Macro_Corner	87.5 × 225	TL	TR

Register = DAC_PREAMP_<letter>_LIN. The mapping and the long rows/columns come from sensor_geometry.yaml.

The file map

Inputs (you bring)

- `CMSIT.xml`
- `my_mask_02.txt`
- `my_mask_13.txt`
- `H0_ROC0-3.txt`
- `sensor_geometry.yaml`

Scripts

- `auto_align_parabola.py`
- `scurve_launch.py`
- `scurve_graph_parabola.py`
- `parabola_fit.py`
- `timewalk_common.py`
- `test_parabola_fit.py`

Outputs (made by runs)

- `Results/*.root`
- `scan_runs.yaml`
- `positions.yaml`
- `Plots_parabola/`
- `curves/`
- `auto_align_*.log`

Every path is relative: the scripts chdir into their own folder first, so inputs, scripts and outputs all live in one directory.

One alignment iteration

1 auto_align_parabola.py
Enables the target chips, starts a scan

→

2 scurve_launch.py
Per delay: set it in the XML, run CMSITminiDAQ -c scurve

→

3 Results/ + scan_runs.yaml
One ROOT file per run, plus the run → delay list

4 scurve_graph_parabola.py
Builds a curve per geometry, calls parabola_fit

→

5 positions.yaml
pos, err, edge for every chip and geometry

→

6 auto_align_parabola.py
Shift vs. Standard → next DAC → writes CMSIT.xml

Solid box = script, dashed box = file. Steps 1 and 6 are the same script. Step 6 goes back to step 2 until every geometry is aligned or **MAX_ITERATIONS** is reached. Steps 2 and 4 run as subprocesses.

Both subprocesses share the scan settings NUM_SCANS, RUNS_PER_DELAY and DELAY_STEP_SIZE from timewalk_common.py, passed as arguments.

`timewalk_common.py`

Shared settings and helpers

What it holds

- **All search and scan settings**: MAX_ABS_SHIFT, MAX_ITERATIONS, DRIFT_SIGMA, slopes, step caps, register limits, NUM_SCANS, RUNS_PER_DELAY, DELAY_STEP_SIZE, TARGET_CHIPS, pixel areas
- **enter_workdir()**: chdir into the script folder
- **XML**: enable chips, read and write registers
- **GeometryManager**: row, col → Standard / Long_Row / Long_Col / Macro_Corner
- **Masks**: active pixels from my_mask_02 / 13
- **extract_pixel_data()**: threshold and noise per pixel from a ROOT file

Connections

Imported by
auto_align_parabola.py
scurve_graph_parabola.py

Reads
CMSIT.xml, sensor_geometry.yaml, mask files, Results/*.root

Writes
CMSIT.xml (DACs, chip enable), the log

`scurve_launch.py`

The DAQ loop

What it does

1. For each delay 0, step, … below NUM_SCANS: write CAL_EDGE_FINE_DELAY into the XML
2. Run CMSITminiDAQ -c scurve RUNS_PER_DELAY times
3. Compare the Results/ listing before and after each run: new ROOT files = that delay's runs
4. On a crash or a 180 s timeout, skip the rest of that delay
5. Write the run → delay list to scan_runs.yaml

Connections

Called by
auto_align_parabola.py (subprocess)

Reads / writes
CMSIT.xml (delay only)

Produces
Results/RunNNNNNN_SCurve.root
scan_runs.yaml

Imports
nothing local

`scurve_graph_parabola.py`

From ROOT files to positions

What it does

1. Run list from scan_runs.yaml (or count from --start_run)
2. Per run and chip: masked pixels, typed by geometry
3. Per geometry and delay: mean threshold; error = scatter between repeat runs / √repeats
4. parabola_fit gives pos, err, edge for each curve
5. Write positions.yaml and the plots

Offline: **--curves_only** saves curves/*.yaml, **--from_curves** refits them without ROOT files.

Connections

Called by
auto_align_parabola.py (subprocess)

Imports
parabola_fit, timewalk_common

Reads
Results/, scan_runs.yaml, masks, geometry

Writes
positions.yaml, Plots_parabola/

parabola_fit.py

How one curve gets a position

1 Flat?

Highest – lowest below MIN_AMPLITUDE: **rejected**, DAC not moved

2 Dip inside?

Fewer than EDGE_POINTS on one side of the minimum: no position, only **edge** = left (too early) or right (too late)

3 Parabola

Least-squares fit to PARABOLA_POINTS around the lowest point. **Vertex = position**

4 Monte Carlo

N_MC fakes, each point jiggled by its error. **Spread = err**

A curve is rejected if err is above MAX_POS_ERR or more than MC_MAX_FAIL of the fakes fail. The window is wide on purpose: the dip bottom is flat, so the flanks pin the vertex down. It is pure numpy, so **test_parabola_fit.py** can check it offline.

`auto_align_parabola.py`

Choosing the next DAC

1. **Reference** = the geometry with the smallest area (Standard)
2. **Shift** = target pos − reference pos; aligned if |shift| ≤ MAX_ABS_SHIFT
3. **Secant step**: the line through two (DAC, shift) points, preferring one on each side of zero; the next DAC is where it crosses zero
4. No usable slope: **DEFAULT_SLOPE**. No dip: step **NO_DIP_STEP** in the edge direction
5. Step capped at **MAX_DAC_STEP**, always at least 1, kept in 0–1023

Sign rule

Higher DAC = later curve.

Shift > 0 (too late): lower the DAC.

Shift < 0 (too early): raise it.

All targets on all chips move together, then one shared scan measures them all.

`auto_align_parabola.py`

After every scan

What the scan shows	What the aligner does
Moved geometry has a dip	Adds (DAC, shift) to its history; converged if aligned
Moved geometry has no dip	Steps toward its edge direction next time
Moved geometry flat or too uncertain	Search abandoned (can re-open later)
Aligned one now just past the limit	Within DRIFT_SIGMA error bars: noise, stays OK
Aligned one clearly drifted out	Re-opened, earlier points kept
Reference lost its dip	That chip stops, DACs kept

Every geometry is re-measured on every scan. The final table and SUCCESS come from the **last** scan.

Running it

Setup

- The 6 files plus CMSIT.xml, the masks and H0_ROC*.txt, all in one folder, with these exact names
- Ph2_ACF set up: CMSITminiDAQ on the PATH, PH2ACF_BASE_DIR
- Python with PyROOT, numpy, pandas, matplotlib, pyyaml

Commands

```
python3 test_parabola_fit.py
python3 auto_align_parabola.py
```

The test is offline and safe. The aligner drives the hardware: back up CMSIT.xml first. Output goes to auto_align_parabola_<timestamp>.log.